

COMPARATIVE ANALYSIS

1. Direct Addressing v.s Hashing

Basis	Direct Addressing	Hashing
Memory Requirement	Requires a table for the entire key range, so it can waste a lot of memory	Requires a smaller table based mainly on the no. of stored element
Flexibility	Less flexible; mainly suitable for small, bounded integer keys	More flexible; can handle large ^{keys} using hash function
Real-world applicability	Useful when the key range is small and known	Widely useful for dictionaries, databases, symbol tables, caches
Search time	$O(1)$ worst case	$O(1)$ average with good hash function but can become $O(n)$ in the worst case because of collisions

2. Load factor in Hashing vs. Input Size in Sorting

Basis	Load factor in hashing	Input size in Sorting
Meaning	Load factor, where number of elements and m : table size	Input size: no. of elements that must be sorted
Performance degradation	As load factor increases, Collision increase and hash-table operations become slower	As input size increases the number of comparisons/operations increase
Effect	High load factor can cause searching to approach $O(n)$	For eg: Bubble sort can reach $O(n^2)$, while merge sort are $O(n \log n)$
Optimization Strategies	Increase table size, rehash, use a good hash function	Choose an efficient sorting algorithm, reduce unnecessary comparisons